SLE-4: FINAL VIVA & ARCHITECTURE DECISION RECORD

Course: 02AML204 – Introduction to Artificial Intelligence
S.Y. B.Tech. CSE (AI & ML)

Name	Pranali Nandkishor More	**PRN**	25UAM132
Division	B	**Date**	11 October 2026

PART A: ARCHITECTURE DECISION RECORD (ADR)

Title: Implementing and Comparing BFS and DFS for Graph Search

1. Context

The project searches a graph from a start node to a goal node. Breadth-First Search (BFS) and Depth-First Search (DFS) are uninformed search algorithms. BFS explores level by level using a queue (FIFO). DFS explores one path deeply before backtracking, using a stack or recursion (LIFO). The project work covers implementation, performance profiling and C4 architecture diagrams.

2. Decision

I decided to implement and compare BFS and DFS on the same graph, using the same start node and selected goal positions. The comparison considers average execution time and nodes expanded. This provides an evidence-based comparison rather than assuming one algorithm is always superior.

3. Alternatives Considered

• **BFS only:** Finds a shortest path in an unweighted graph, but does not give a direct comparison with DFS.

• **DFS only:** Can explore deep paths and may use less memory, but does not guarantee a shortest path.

• **A* search:** Uses a heuristic to guide search. It was not selected because designing and validating a heuristic was outside the intended scope.

4. Rationale / Justification

Both algorithms were implemented because they have different strengths. BFS is useful when shortest path length matters in an unweighted graph. DFS is useful for deep exploration and may use less memory. Testing both with identical inputs makes the comparison fair.

Performa

Test case	Goal	BFS avg. time (ms)	DFS avg. time (ms)	BFS nodes	DFS nodes
Best case	B	0.019620	0.004310	2	2
Average case	I	0.020910	0.013620	9	6
Worst case	K	0.126880	0.022080	11	11

Note: Performance values are copied from the provided sample report; verify against your own SLE-2 measurements.

Note: Fill this chart using your own measured results. The sample report belongs to another student, so their timing values and repository information are not presented as yours.

5. Consequences

Positive: Both algorithms can be compared under consistent conditions; the modular design is easier to explain and extend.

Trade-offs: Maintaining two algorithms requires extra code and testing. BFS may use more memory for a wide frontier. DFS may return a non-shortest path. Runtime depends on graph structure, node ordering and implementation.

PART B: SHORT SUMMARY OF MY SLE JOURNEY

In SLE-1, I worked on BFS and DFS search code and related project files. In SLE-2, I profiled the programs and compared execution time and nodes expanded. In SLE-3, I represented the system using C4 Context, Container, Component and Code diagrams. In SLE-4, I record the decision to compare both algorithms and connect performance evaluation with modular architecture.

This journey connects four skills: writing search code, measuring performance, designing system architecture and explaining technical decisions.

PART C: AI CONTRIBUTION NOTE

AI tools used: Large language models as a learning and drafting aid.

What AI helped with: Organising the ADR, clarifying concepts and preparing possible viva questions.

What I did myself: I am responsible for running and checking the code, verifying measurements against my SLE-2 report, confirming architecture details against my SLE-3 diagrams and explaining the work in my own words.

PART D: VIVA PREPARATION NOTES

1. What is BFS? Breadth-First Search explores nodes level by level using a queue (FIFO). In an unweighted graph, it finds a shortest path.

2. What is DFS? Depth-First Search follows one branch deeply before backtracking. It uses a stack or recursion and does not guarantee the shortest path.

3. Why compare BFS and DFS? To understand practical differences in execution time, nodes expanded, memory use and path quality.

4. What is an ADR? An Architecture Decision Record documents the context, decision, alternatives, justification and consequences of a design choice.

5. What is C4 architecture? It describes a system at four levels: Context, Container, Component and Code.

6. What is the main difference between BFS and DFS? BFS searches level by level and can find a shortest path in an unweighted graph; DFS explores deeply and backtracks.

7. How do you measure performance? Repeat timing runs, compare average execution time and count nodes expanded inside the search functions. Use the same graph and start node for both algorithms.

8. What if the graph becomes larger? Runtime and memory requirements may increase. BFS may store a wide frontier; recursive DFS may reach recursion-depth limits.

9. What did AI contribute? AI helped organise the report and clarify wording. The code, measurements and final claims must be checked and understood by me.

10. Summarise your decision. I compared BFS and DFS to understand their practical trade-offs using the same graph and performance measures.

FINAL CHECKLIST BEFORE UPLOAD & VIVA

- Enter your verified SLE-2 performance values in the chart.
- Confirm the GitHub repository and architecture details match your actual work.
- Follow the required file name and page limit.
- Be prepared to explain the algorithms and results in your own words.